BDA Week 13-14 — Clustering: k-Means, Hierarchical, BFR, CURE

Module 2 of Phase-1 Final Prep · Numerical-trace heavy topic

Module 2 of Phase-1 Final Prep · Numerical-trace heavy topic

Topics covered in this PDF:

• Divisive (top-down): start with one cluster; recursively split

CS-404 Big Data Analytics · Dr. Syed Imran Ali · Final ~ 2026-05-16

Prepared as a comprehensive professor-grade study tutor.

Contents

1. §1 — Beginning Key Notes (Study Compass)
2. §2 — What is Clustering? Distance Measures
3. §3 — k-Means (Lloyd's Algorithm)
4. §4 — Hierarchical (Agglomerative) Clustering
5. §5 — BFR (Bradley-Fayyad-Reina)
6. §6 — CURE (Clustering Using REpresentatives)
7. §7 — Comparison Table
8. §8 — Six Worked Numerical Examples
9. §9 — Practice Questions (15)
10. §10 — Full Worked Answers
11. §11 — Ending Key Notes (Revision Cards)
12. §12 — Formula & Algorithm Reference

Why this PDF is critical.

The midterm tested CURE clustering with a numerical 2D dataset. The final almost certainly recycles the pattern with a NEW dataset and probably probes BFR Mahalanobis arithmetic too. This examiner does not ask "define hierarchical clustering" — he asks "merge these 5 points step by step using single-linkage and draw the dendrogram." Master §3–§8 here and 25–35 marks are locked in.

§1 — Beginning Key Notes (Study Compass)

These are the ten load-bearing ideas you must walk into the exam owning. Every numerical clustering question reduces to applying one of them.

1. **Cluster = group with high intra-similarity, low inter-similarity.** Members of one cluster are close to each other; members of different clusters are far apart. The whole game is choosing what *close* means.
2. **Distance toolbox:** Euclidean (L_2) for points in space, Cosine for vector direction (text), Jaccard for sets (CDs/customers), edit distance for strings. The choice of distance literally changes the answer.
3. **Curse of dimensionality.** In high-d, almost all pairs of random points are at the *same* distance. Clusters that look obvious in 2D become invisible in 1000D — this is the hidden enemy in every real BDA problem.
4. **Two algorithm families. Hierarchical** (agglomerative bottom-up, divisive top-down) builds a dendrogram by merging. **Point-assignment** (k-means, BFR, CURE) maintains k clusters and reassigns points.
5. **k-Means / Lloyd's algorithm.** Pick k, place initial centroids, assign each point to nearest centroid, recompute centroids as means, repeat to convergence. Output depends on initialization.
6. **Hierarchical linkage choices** define cluster-to-cluster distance differently: **single** = min pair-distance (loose chains), **complete** = max pair-distance (compact balls), **average** = mean pair-distance, **centroid** = distance between centroids. Same data → very different dendrograms.
7. **BFR for huge data.** Bradley-Fayyad-Reina is k-means rewritten so the data lives on disk. We never store points — only **summary statistics** *(N, SUM, SUMSQ)* per cluster — that's $2d+1$ numbers per cluster.
8. **Three BFR sets: Discard Set (DS)** — points assigned to a cluster and replaced by stats. **Compression Set (CS)** — mini-clusters of points close to each other but not to any DS centroid. **Retained Set (RS)** — lonely outliers waiting.
9. **Mahalanobis distance** is the BFR membership test: $d_M(x,c) = \sqrt{\Sigma_i ((x_i - c_i)/\sigma_i)^2}$. Threshold ≈ $\sqrt{d}$ to $2\sqrt{d}$ — accept point if d_M below threshold.
10. **CURE for non-convex clusters.** Sample → hierarchical-cluster the sample → for each cluster pick k scattered representatives → shrink each rep α\% ($\alpha \approx 0.2$) toward the centroid → assign every remaining point to the nearest representative.

The single biggest exam pattern.

Examiner gives you 5–8 points in 2D and asks you to walk a chosen algorithm through every iteration. Show every distance computation, every assignment, every centroid update. Master *that* procedure across all four algorithms and you cannot lose more than a handful of marks.

§2 — What is Clustering? Distance Measures

Setup. Given a set of points $\{x_1, x_2, \ldots, x_n\}$ with a notion of distance $d(x_i, x_j)$, partition the points into groups (clusters) such that intra-cluster distance is small and inter-cluster distance is large.

Analogy — sorting laundry.

Imagine a heap of mixed clothes. You don't have labels; you only have your sense of "these two T-shirts look more alike than that T-shirt and that sock." Clustering is the same: no labels, just a similarity sense, and you separate the heap into bins.

2.1 Distance measures

The choice of distance is *the* most important design decision. Same dataset can produce different clusters under different distances.

Euclidean (L_2) distance.

$$d(x, y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

Standard for 2D/3D physical space. Used in k-means, BFR, CURE.

Cosine distance.

$$d_{\cos}(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$$

Measures the *angle* between two vectors, ignoring magnitude. Standard for text/document vectors where direction (which words appear) matters more than length (how many words).

Jaccard distance.

$$d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

For *sets*. Two CDs are similar if their customer-sets overlap heavily.

Edit (Levenshtein) distance. Minimum number of single-character insertions, deletions, or substitutions to convert one string into another. Used for sequence/text clustering.

Domain	Right distance
2D/3D physical points	Euclidean
Document = word-vector	Cosine

Domain	Right distance
Document = word-set / CDs = customer-set	Jaccard
DNA / strings	Edit

2.2 The curse of dimensionality

In 2D our intuition works. In 10 000-D it breaks. Two facts:

1. **Concentration of distances.** For random points uniformly distributed in $[0,1]^d$, the ratio (max pairwise distance) / (min pairwise distance) tends to 1 as $d \to \infty$. Almost all pairs are at the *same* distance.
2. **Volume on the surface.** In high d, almost all the volume of a unit hypercube lies within a thin shell at the surface. There is no "centre" for clusters to inhabit.

Consequence. Naive distance-based clustering becomes meaningless in raw high-dimensional space. We either reduce dimensions (PCA, hashing) or use cluster algorithms specifically tuned for high-d — exactly why BFR and CURE matter for real BDA.

EXAM TRAP — "obvious" clusters in 2D.

Slides show 2D blobs that any human eye can group. The exam dataset will also be 2D (so you can compute by hand) but in real life the same algorithm runs in $d = 10^4$. Quote both facts in conceptual answers; show 2D arithmetic in numerical answers.

2.3 Two algorithm families

Hierarchical clustering. Builds a tree (dendrogram) of nested clusters.

- *Agglomerative* (bottom-up): start with each point as its own cluster; repeatedly merge the two nearest clusters until one big cluster remains.
- *Divisive* (top-down): start with one cluster; recursively split.

Point-assignment clustering. Maintains a fixed set of k clusters; iteratively reassigns points.

- *k-means / Lloyd's algorithm:* simple, fast, Euclidean.
- *BFR:* k-means scaled to disk-resident data via summary stats.
- *CURE:* k-means-like but uses multiple representatives per cluster to capture non-convex shapes.

---

§3 — k-Means (Lloyd's Algorithm)

The simplest, most-used clustering algorithm in industry. Assumes Euclidean space and Euclidean distance.

3.1 The algorithm

Inputs: n points in $mathbbR^d$, integer k. **Output:** k clusters with centroids $c_1, ldots, c_k$.

1. **Initialize.** Pick k initial centroids (see §3.2 for choices).

2. **Assignment step.** For each point x, assign x to the cluster whose centroid is nearest:

$$\text{cluster}(x) = \arg\min_{j \in \{1, \ldots, k\}} \|x - c_j\|_2$$

1. **Update step.** For each cluster j, recompute centroid as the mean of its assigned points:

$$c_j \leftarrow \frac{1}{|C_j|} \sum_{x \in C_j} x$$

1. **Repeat** steps 2–3 until **convergence** (no points change clusters and centroids stop moving).

3.2 Initialization choices

The output of k-means depends critically on the starting centroids. Common strategies:

Strategy	How
Random points	Pick k data-points uniformly at random.
Random + farthest	Pick first random; pick each subsequent as far as possible from the previous chosen points.
k-means++	Probabilistic farthest-point: probability $\propto d^2$ to nearest chosen.
Sample then cluster	Take a small sample, optimally cluster it, use those centroids.

The "random + farthest" rule is what the BFR slides use; quote it in concept questions.

3.3 Choosing k — the elbow method

If k is unknown, run k-means for $k = 1, 2, 3, ldots$ and plot **average distance to centroid** versus k. The plot drops steeply, then flattens. The "elbow" is the right k.

- k too small → many points far from their centroid → average distance large.
- k around right → distances drop sharply.
- k too large → little improvement; you are over-fitting.

3.4 Convergence and complexity

- **Convergence guaranteed** to a local minimum (the within-cluster sum of squares decreases monotonically and is bounded below by 0). But the local minimum may not be the global one — initialization matters.
- **Complexity per iteration:** $O(n\ k\ d)$ — every point computes distance to every centroid in every iteration.
- Typical iterations to convergence: 10–30 on well-separated data.

EXAM TRAP — centroid is artificial.

A k-means centroid is the *mean* of points in the cluster — generally **not** one of the data-points. Some texts call the closest-data-point the **medoid** or **clustroid**. Centroid = artificial average; medoid/clustroid = actual data point closest to all others. The exam may ask you to distinguish.

§4 — Hierarchical (Agglomerative) Clustering

Builds a tree of merges from the bottom up. No need to specify k in advance — you read k off the dendrogram by cutting at the right level.

4.1 The algorithm

1. Start: every point is its own singleton cluster. We have n clusters.
2. Compute the $n \times n$ distance matrix between every pair of points.
3. **Merge** the two closest clusters into one.
4. Update the distance matrix: replace the two merged rows/columns with one row/column of distances from the new merged cluster to every other cluster, computed via a **linkage rule**.
5. Repeat 3–4 until only one cluster remains (or stopping criterion is met).
6. Output the **dendrogram** — a binary tree showing every merge and the distance at which it occurred.

4.2 The three big linkage choices

When merging clusters A and B, the *cluster-to-cluster* distance is some function of the *point-to-point* distances between members.

Linkage	*D(A, B)*	Tendency				
Single	$min_{a \in A,\, b \in B}\, d(a, b)$	Long, snaky chains. Sensitive to noise. Connects nearby clusters via single bridges.				
Complete	$max_{a \in A,\, b \in B}\, d(a, b)$	Tight, compact, ball-like clusters. Resistant to chaining.				
Average	$\frac{1}{	A		B	} \sum_{a, b} d(a, b)$	Compromise.
Centroid	$d(\bar{a}, \bar{b})$ where $\bar{}$ = mean	Euclidean only.				

EXAM TRAP — "merge the two closest clusters" depends on linkage.

Two clusters that are *closest* under single-linkage (because one pair is touching) may be far apart under complete-linkage (because their farthest pairs are remote). Always state the linkage you are using before you merge.

4.3 Cluster representation: centroid vs clustroid

In a Euclidean space we represent a cluster by its **centroid** — the mean of its data-points. The centroid is generally not a data-point itself.

In a non-Euclidean space (sets, strings) "average" is undefined. We instead keep the **clustroid** — the data-point most central to the cluster, defined by one of:

- smallest *maximum* distance to any other point in the cluster,
- smallest *average* distance,
- smallest *sum of squared* distances.

4.4 When to stop merging

- A **target *k*** — stop when k clusters remain.
- A **distance threshold** — stop when the next merge would join clusters whose distance exceeds θ.
- A **cohesion threshold** — stop merging when the merged cluster's diameter, average inter-point distance, or density would drop below acceptable.

4.5 Complexity

- Naive: $O(n^3)$ — at each of $n-1$ merges we recompute all pairwise cluster distances.
- Priority-queue based: $O(n^2 \log n)$.
- Both still infeasible for billion-point data — that is what BFR and CURE are for.

---

§5 — BFR (Bradley-Fayyad-Reina)

BFR is k-means rebuilt for the case where the data is too large to fit in memory and lives on disk. Read one chunk at a time, summarize, throw away the points, keep summarising the next chunk. Memory footprint is $O(k \cdot d)$ — *independent of dataset size*.

5.1 Assumptions

- Euclidean space, Euclidean distance.
- Each cluster is **normally distributed** around its centroid.
- Standard deviations may differ per dimension, but axes are **fixed** — clusters are *axis-aligned ellipses*. (Tilted ellipses would need a full $d \times d$ covariance matrix; BFR cheats by assuming no tilt.)

5.2 Three sets of points

At any time, every point seen so far is in exactly one of:

Set	Meaning
Discard Set (DS)	Points close enough to a known cluster centroid that we replaced them with summary stats and threw the points away.

Set	Meaning
Compression Set (CS)	Mini-clusters of points that are close to each other but **not close to any DS centroid**. Stored by summary stats too, but not yet associated with a main cluster.
Retained Set (RS)	Truly isolated points (outliers) that are close to nothing. Kept as raw points.

5.3 Summary statistics (N, SUM, SUMSQ)

For each cluster (DS or CS) we store $2d + 1$ numbers — *not the points*:

- N — number of points in the cluster.
- $SUM = (SUM_1, ldots, SUM_d)$, where $SUM_i = \Sigma_{x \in C} x_i$.
- $SUMSQ = (SUMSQ_1, ldots, SUMSQ_d)$, where $SUMSQ_i = \Sigma_{x \in C} x_i^2$.

From these we can reconstruct:

$$\text{centroid}_i = \frac{\text{SUM}_i}{N}, \qquad \text{variance}_i = \frac{\text{SUMSQ}_i}{N} - \left(\frac{\text{SUM}_i}{N}\right)^2, \qquad \sigma_i = \sqrt{\text{variance}_i}$$

These three numbers — for each cluster, in each dimension — are *all* the algorithm ever needs.

Why $2d+1$?

Storing the full covariance matrix would require $d(d+1)/2$ numbers per cluster. For $d = 1000$ that is half a million per cluster. Forcing axis-aligned ellipses cuts it to $2d + 1 = 2001$ — a 250× saving.

5.4 The Mahalanobis test (Q1: how do we add a new point?)

When a new point x arrives, compute its **Mahalanobis distance** to each cluster's centroid:

$$d_M(x, c) = \sqrt{\sum_{i=1}^{d} \left(\frac{x_i - c_i}{\sigma_i}\right)^2}$$

This is the Euclidean distance *normalized per dimension by the cluster's own spread*. A point 3 standard deviations out in a tight dimension is "farther" than a point 3 standard deviations out in a stretched dimension, in the right normalized sense.

Threshold rule. If clusters are normally distributed in d dimensions, then after Mahalanobis transformation one standard deviation equals $\sqrt{d}$. So:

- Accept the point into the nearest cluster (move it to the DS) if $d_M(x, c) < T \cdot \sqrt{d}$ for some threshold T (commonly $T = 2$ or $T = 3$).
- Otherwise the point goes to either the RS (if no other points are near) or potentially seeds a new CS mini-cluster.

EXAM TRAP — Mahalanobis is normalized.

Don't confuse d_M with the raw Euclidean distance. The $/\sigma_i$ in each term is what makes axis-aligned-elliptical clusters work. If you forgot the σ_i on the exam, you wrote the wrong formula and lost the marks.

5.5 Merging CS sub-clusters (Q2: should two compressed sets combine?)

Compute the variance of the *combined* sub-cluster (cheap with N, SUM, SUMSQ — additive). Combine if the combined variance falls below a threshold. Else leave them separate.

The additivity is the magic: if cluster C has stats $(N_C, SUM_C, SUMSQ_C)$ and cluster C' has $(N_{C'}, SUM_{C'}, SUMSQ_{C'})$, then the merged cluster has:

$$N_{C \cup C'} = N_C + N_{C'}, \quad \mathrm{SUM}_{C \cup C'} = \mathrm{SUM}_C + \mathrm{SUM}_{C'}, \quad \mathrm{SUMSQ}_{C \cup C'} = \mathrm{SUMSQ}_C + \mathrm{SUMSQ}_{C'}$$

No need to revisit the points.

5.6 The full BFR algorithm

```
Initialize: pick k initial centroids from the first memory-load.
for each subsequent memory-load of points:
for each point x:
compute Mahalanobis distance to each DS centroid
if min d_M < T·sqrt(d): assign x to that DS cluster
update N, SUM, SUMSQ for that cluster
else: x stays as raw → goes into the "leftover" pool
cluster the leftover pool ∪ old RS using main-memory clustering:
clusters with >1 point → CS (store stats)
single isolated points → RS
consider merging CS sub-clusters whose combined variance < threshold
on the final memory-load:
assign every CS cluster and RS point to its nearest DS cluster
```

§6 — CURE (Clustering Using REpresentatives)

BFR fails on non-spherical clusters: a long crescent, two interlocking S-curves, a ring around a blob. CURE fixes this by representing each cluster with **multiple scattered representative points** instead of one centroid — and shrinking those reps slightly toward the centroid to dampen outlier sensitivity.

6.1 What CURE buys you over BFR

Cluster shape	BFR works?	CURE works?
Spherical / axis ellipse	Yes	Yes
Tilted ellipse	No	Yes
Long thin curve	No	Yes
Concentric rings	No	Yes (separate rings as long as gap is enough)

6.2 Phase 1 — pick scattered representatives, shrink them

1. **Sample.** Pick a random sample of points that fits in main memory.
2. **Hierarchical-cluster the sample.** Run agglomerative clustering on the sample to get *k* initial clusters.
3. **Pick representatives.** For each cluster, iteratively pick ***r* scattered points** — typically by the *farthest-point* rule:
   - Pick the point farthest from the cluster's centroid as the first representative.
   - Pick each subsequent representative as the point farthest from the already-chosen representatives (max-min distance).
   - Continue until you have *r* representatives (typical *r = 4* to *10*).
1. **Shrink toward the centroid.** For each representative *p* and the cluster centroid *c*, replace *p* with:

$$p' = p + \alpha \cdot (c - p)$$

where $\alpha \in [0, 1]$ is the shrink factor (typical $\alpha = 0.2$). Each rep moves *20\%* of the way toward the centroid. This kills outlier-induced rep positions while still letting reps fan out across the cluster's shape.

6.3 Phase 2 — assign all remaining points

Rescan the entire (possibly disk-resident) dataset. For each point *p*:

1. Compute Euclidean distance from *p* to every representative of every cluster.
2. Assign *p* to the cluster whose **representative is closest**.

That is the entire second pass. Note: closest **representative**, not closest **centroid**. Multiple reps per cluster are what let CURE wrap around non-convex shapes.

> **EXAM TRAP — α direction.**
>
> Shrinking moves the rep *toward* the centroid (inward), not away. A rep that started at the cluster's edge now sits a bit inside. Forgetting this means you compute *p' = p - α(c-p)* and your reps fly outward — wrong.

6.4 Why representatives + shrinkage handle non-convex clusters

A single centroid (BFR) cannot fit a curved cluster — the centroid is in the curve's "hollow", and points on the far ends of the curve become farther from the centroid than points in completely different clusters. With *r* representatives spread across the curve, the farthest point on one end is still close to *one* of its cluster's reps. The shrinkage prevents an outlier from being picked as a rep so far out that it "captures" points belonging to a neighbour.

---

§7 — Comparison Table

Property	k-Means	Hierarchical Agglomerative	BFR	CURE
Memory footprint	$O(n + kd)$	$O(n^2)$ (distance matrix)	$O(kd)$ (just stats)	$O(sample + krd)$
Scales to billions of points	OK if in RAM	No (n^2)	Yes (designed for it)	Yes
Cluster shapes handled	Spherical	Any (depends on linkage)	Axis-aligned ellipses	Arbitrary convex/non-convex
Distance type	Euclidean only	Any (Euclidean / Jaccard / cosine)	Euclidean only	Euclidean only
Needs k in advance	Yes	No (cut dendrogram)	Yes	Yes
Deterministic?	No (init-dep.)	Yes (given linkage)	No (init-dep.)	No (sample- and init-dep.)
Complexity	$O(nkdT)$	$O(n^2 \log n)$	$O(n)$ I/O + RAM-bounded compute	Two passes over data
Output form	k centroids	Dendrogram	k DS-stats + CS + RS	k clusters with r reps each

§8 — Six Worked Numerical Examples

Every example below is the kind of multi-step trace your examiner expects. Read once, close the document, reproduce on paper.

Worked Example 1 — k-Means with 6 points, $k = 2$, three iterations

Problem. Six 2D points: $A=(1,1)$, $B=(2,1)$, $C=(1,2)$, $D=(8,8)$, $E=(9,8)$, $F=(8,9)$. Run k-means with $k=2$ starting from initial centroids $c_1^{(0)} = A = (1,1)$, $c_2^{(0)} = D = (8,8)$. Use Euclidean distance.

Iteration 1 — Assignment. Compute distance from every point to each centroid; assign to nearer.

Point	$d(\cdot, c_1=(1,1))$	$d(\cdot, c_2=(8,8))$	Assigned
A=(1,1)	0	$\sqrt{49+49}=9.90$	C1
B=(2,1)	$\sqrt{1+0}=1.00$	$\sqrt{36+49}=9.22$	C1
C=(1,2)	$\sqrt{0+1}=1.00$	$\sqrt{49+36}=9.22$	C1
D=(8,8)	9.90	0	C2
E=(9,8)	$\sqrt{64+49}=10.63$	$\sqrt{1+0}=1.00$	C2

Point	$d(\cdot, c_1=(1,1))$	$d(\cdot, c_2=(8,8))$	Assigned
F=(8,9)	$\sqrt{49+64}=10.63$	$\sqrt{0+1}=1.00$	C2

Cluster C1 = *A, B, C*\, Cluster C2 = *D, E, F*\.

Iteration 1 — Update.

$$c_1^{(1)} = \left(\frac{1+2+1}{3}, \frac{1+1+2}{3}\right) = \left(\frac{4}{3}, \frac{4}{3}\right) \approx (1.33, 1.33)$$

$$c_2^{(1)} = \left(\frac{8+9+8}{3}, \frac{8+8+9}{3}\right) = \left(\frac{25}{3}, \frac{25}{3}\right) \approx (8.33, 8.33)$$

Iteration 2 — Assignment. With centroids *(1.33, 1.33)* and *(8.33, 8.33)*, every point clearly stays in its previous cluster (the gap is huge). No reassignments.

Iteration 2 — Update. No points moved → centroids unchanged.

Iteration 3. No change → **converged** at $r=2$.

Final answer. Clusters: *A, B, C*\ centred at *(1.33, 1.33)* and *D, E, F*\ centred at *(8.33, 8.33)*.

Lesson. Well-separated data converges in ~ 2 iterations. The exam may use less-clean data so you actually need 3 iterations to stabilise.

Worked Example 2 — Hierarchical agglomerative, single-linkage

Problem. Five 2D points: $P_1=(1,1)$, $P_2=(2,1)$, $P_3=(4,3)$, $P_4=(5,4)$, $P_5=(9,9)$. Cluster bottom-up using **single-linkage** Euclidean distance. Show every merge with the distance matrix update.

Step 0 — initial pairwise distances.

	P1	P2	P3	P4	P5
P1	0	1.00	3.61	5.00	11.31
P2		0	2.83	4.24	10.63
P3			0	1.41	7.81
P4				0	6.40
P5					0

Computations: $d(P_1,P_2)=\sqrt{1^2+0^2}=1$; $d(P_1,P_3)=\sqrt{3^2+2^2}=\sqrt{13}\approx 3.61$; $d(P_3,P_4)=\sqrt{1^2+1^2}=\sqrt{2}\approx 1.41$; $d(P_4,P_5)=\sqrt{4^2+5^2}=\sqrt{41}\approx 6.40$ and so on.

Merge 1. Smallest entry is $d(P_1,P_2) = 1.00$. Merge \P_1, P_2\ at height *1.00*.

Update matrix (single-linkage). $d(\backslash P_1,P_2\backslash, P_k) = \min(d(P_1,P_k), d(P_2,P_k))$:

	{P1,P2}	P3	P4	P5
{P1,P2}	0	2.83	4.24	10.63
P3		0	1.41	7.81
P4			0	6.40

	{P1,P2}	P3	P4	P5
P5				0

(Row {P1,P2}: *min(3.61, 2.83) = 2.83* for P3; *min(5.00, 4.24) = 4.24* for P4; *min(11.31, 10.63) = 10.63* for P5.)

Merge 2. Smallest is $d(P_3, P_4) = 1.41$. Merge $\{P_3, P_4\}$ at height *1.41*.

	{P1,P2}	{P3,P4}	P5
{P1,P2}	0	2.83	10.63
{P3,P4}		0	6.40
P5			0

(Row {P1,P2} to {P3,P4}: *min(2.83, 4.24) = 2.83*. Row {P3,P4} to P5: *min(7.81, 6.40) = 6.40*.)

Merge 3. Smallest is $d(\{P_1,P_2\}, \{P_3,P_4\}) = 2.83$. Merge into $\{P_1,P_2,P_3,P_4\}$ at height *2.83*.

	{P1,P2,P3,P4}	P5
{P1,P2,P3,P4}	0	6.40
P5		0

(Single-linkage: *min(10.63, 6.40) = 6.40*.)

Merge 4. Final merge $\{P_1,\ldots,P_4\} \cup \{P_5\}$ at height *6.40*.

Dendrogram heights: *1.00 → 1.41 → 2.83 → 6.40*.

```
6.40
+------+------+
2.83 |
+----+----+ |
1.00 1.41 |
+-+-+ +-+-+ |
P1 P2 P3 P4 P5
```

Cutting the dendrogram. Cut at height *5* → 2 clusters: $\{P_1,P_2,P_3,P_4\}$ and $\{P_5\}$. Cut at height *2* → 3 clusters: $\{P_1,P_2\}$, $\{P_3,P_4\}$, $\{P_5\}$.

Worked Example 3 — Same data, complete-linkage

Problem. Same five points. Use **complete-linkage** instead.

Merge 1. Same starting matrix → smallest pair is still $d(P_1,P_2)=1.00$. Merge at height *1.00*.

Update (complete-linkage). $d(\{P_1,P_2\}, P_k) = \max(d(P_1, P_k), d(P_2, P_k))$:

	{P1,P2}	P3	P4	P5
{P1,P2}	0	3.61	5.00	11.31
P3		0	1.41	7.81
P4			0	6.40
P5				0

(Row {P1,P2}: *max(3.61, 2.83) = 3.61* for P3; *max(5.00, 4.24) = 5.00* for P4; *max(11.31, 10.63) = 11.31* for P5.)

Merge 2. Smallest is $d(P_3, P_4)=1.41$. Merge at height *1.41*.

	{P1,P2}	{P3,P4}	P5
{P1,P2}	0	5.00	11.31
{P3,P4}		0	7.81
P5			0

(Row {P1,P2} to {P3,P4}: *max(3.61, 5.00) = 5.00*. Row {P3,P4} to P5: *max(7.81, 6.40) = 7.81*.)

Merge 3. Smallest is $d(\{P_1,P_2\}, \{P_3,P_4\})=5.00$. Merge at height *5.00*.

	{P1,P2,P3,P4}	P5
{P1,P2,P3,P4}	0	11.31
P5		0

(Complete-linkage: *max(11.31, 7.81) = 11.31*.)

Merge 4. Final merge at height *11.31*.

Dendrogram heights (complete): *1.00 → 1.41 → 5.00 → 11.31*.

Compare to single-linkage. Same merge *order* this time, but heights *5.00* vs *2.83* and *11.31* vs *6.40* differ. On more spread-out data the merge order itself can change between linkages — single-linkage is famous for chaining together points connected by single short edges, complete-linkage refuses such loose merges.

Worked Example 4 — BFR Mahalanobis test

Problem. A DS cluster in 2D has summary stats:

- $N = 10$, $SUM = (50, 30)$, $SUMSQ = (260, 100)$.

A new point $x = (5.5, 2.8)$ arrives. Use a $T = 3$ threshold (i.e. accept if $d_M < 3\sqrt{d}$). Should we add x to this cluster?

Step 1 — centroid.

$$c = \left(\frac{\text{SUM}_1}{N}, \frac{\text{SUM}_2}{N}\right) = \left(\frac{50}{10}, \frac{30}{10}\right) = (5, 3)$$

Step 2 — variance per dimension.

$$\sigma_1^2 = \frac{\text{SUMSQ}_1}{N} - \left(\frac{\text{SUM}_1}{N}\right)^2 = \frac{260}{10} - 5^2 = 26 - 25 = 1 \;\Rightarrow\; \sigma_1 = 1$$

$$\sigma_2^2 = \frac{\text{SUMSQ}_2}{N} - \left(\frac{\text{SUM}_2}{N}\right)^2 = \frac{100}{10} - 3^2 = 10 - 9 = 1 \;\Rightarrow\; \sigma_2 = 1$$

Step 3 — Mahalanobis distance.

$$d_M(x, c) = \sqrt{\left(\frac{5.5-5}{1}\right)^2 + \left(\frac{2.8-3}{1}\right)^2} = \sqrt{0.25 + 0.04} = \sqrt{0.29} \approx 0.539$$

Step 4 — threshold. $d = 2$, so threshold $= T\sqrt{d} = 3\sqrt{2} \approx 4.243$.

Step 5 — decision. *0.539 < 4.243* → **accept.** Add *x* to the DS.

Step 6 — update statistics.

- $N \leftarrow 10 + 1 = 11$
- $SUM \leftarrow (50 + 5.5, 30 + 2.8) = (55.5, 32.8)$
- $SUMSQ \leftarrow (260 + 5.5^2, 100 + 2.8^2) = (260 + 30.25, 100 + 7.84) = (290.25, 107.84)$

New centroid: *(55.5/11, 32.8/11) ≈ (5.045, 2.982)* — barely moved.

Sanity check.

$d_M = 0.539$ is well under one σ in *both* dimensions; the point is almost on the centroid. Acceptance was obvious. The exam question may have $\sigma_1 \neq \sigma_2$ (e.g. SUMSQ different) so the dimensions normalise differently — read the SUMSQ carefully.

Worked Example 5 — CURE Phase 1 (representatives + shrinkage)

Problem. A single cluster of five 2D points: *(1,1), (2,1), (1,2), (2,2), (1.5, 1.5)*\. Pick $r = 2$ representatives by farthest-point rule, then shrink each by $\alpha = 0.5$ toward the centroid.

Step 1 — centroid of cluster.

$$c = \left(\frac{1+2+1+2+1.5}{5}, \frac{1+1+2+2+1.5}{5}\right) = \left(\frac{7.5}{5}, \frac{7.5}{5}\right) = (1.5, 1.5)$$

Step 2 — pick first rep: point farthest from centroid.

Point	Distance from *(1.5,1.5)*
(1,1)	$\sqrt{0.25 + 0.25} = \sqrt{0.5} \approx 0.707$
(2,1)	$\sqrt{0.25 + 0.25} \approx 0.707$
(1,2)	$\sqrt{0.25 + 0.25} \approx 0.707$
(2,2)	$\sqrt{0.25 + 0.25} \approx 0.707$
(1.5,1.5)	*0*

Four corners are tied at ≈ *0.707*. Break the tie alphabetically/lexicographically — pick $r_1 = (1, 1)$.

Step 3 — pick second rep: point farthest from $r_1 = (1,1)$.

Point	Distance from *(1,1)*
(2,1)	*1.000*
(1,2)	*1.000*
(2,2)	$\sqrt{1+1} \approx 1.414$
(1.5,1.5)	$\sqrt{0.5} \approx 0.707$

Farthest is *(2, 2)*. So $r_2 = (2, 2)$.

(For the general farthest-point rule with $r > 2$: pick subsequent reps to maximise the *minimum* distance to the previously chosen reps. With $r=2$ the rule reduces to "farthest from r_1".)

Step 4 — shrink each rep by *α = 0.5* toward centroid *c = (1.5, 1.5)*.

For rep *p*, shrunk rep *p' = p + α(c - p)*.

$r_1 = (1,1)$:

$$r_1' = (1, 1) + 0.5 \cdot ((1.5, 1.5) - (1, 1)) = (1, 1) + 0.5 \cdot (0.5, 0.5) = (1.25, 1.25)$$

$r_2 = (2,2)$:

$$r_2' = (2, 2) + 0.5 \cdot ((1.5, 1.5) - (2, 2)) = (2, 2) + 0.5 \cdot (-0.5, -0.5) = (1.75, 1.75)$$

Final shrunk reps: \(1.25, 1.25), (1.75, 1.75)\.

Effect of $\alpha = 0.5$.

Half-shrinkage pulls reps very far inward — they now sit close to the centroid. Real CURE uses *α ≈ 0.2* for milder shrinkage; *α = 0.5* here is just to make the arithmetic clean.

Worked Example 6 — CURE Phase 2 (assign new point)

Problem. Two clusters with shrunk representatives:

- Cluster A reps: *(1.25, 1.25)*, *(1.75, 1.75)* (from WE5).
- Cluster B reps: *(7, 7)*, *(8.5, 8)*.

A new point *p = (3, 2)* arrives. Which cluster does Phase 2 assign it to?

Step 1 — distance to every rep.

Rep	Cluster	*d(p, rep)*
(1.25, 1.25)	A	$\sqrt{(3-1.25)^2 + (2-1.25)^2} = \sqrt{3.0625 + 0.5625} = \sqrt{3.625} \approx 1.904$
(1.75, 1.75)	A	$\sqrt{(3-1.75)^2 + (2-1.75)^2} = \sqrt{1.5625 + 0.0625} = \sqrt{1.625} \approx 1.275$
(7, 7)	B	$\sqrt{(3-7)^2 + (2-7)^2} = \sqrt{16 + 25} = \sqrt{41} \approx 6.403$
(8.5, 8)	B	$\sqrt{(3-8.5)^2 + (2-8)^2} = \sqrt{30.25 + 36} = \sqrt{66.25} \approx 8.139$

Step 2 — pick the closest rep. Smallest is *1.275* at rep *(1.75, 1.75)* in Cluster A.

Step 3 — assign. *p* goes to **Cluster A**.

Compare with a centroid-only (BFR-style) assignment. Centroid of A is *(1.5, 1.5)* (from WE5 step 1), centroid of B (assume) is *(7.75, 7.5)*. $d(p, c_A) = \sqrt{2.25 + 0.25} = \sqrt{2.5} \approx 1.581$, $d(p, c_B) \approx 7.279$. Either way A wins. But in non-convex cases a CURE rep may be much closer than a centroid would have been — that is the whole point.

§9 — Practice Questions (15)

Mix: 6 numerical traces, 5 conceptual short-answer, 4 MCQ / true-false. Time yourself: ~ *6* min per numerical, ~ *3* min per conceptual, ~ *1* min per MCQ.

Q1 [Numerical · 8 marks · Intermediate] ** Five 2D points: $A=(2,2)$, $B=(3,2)$, $C=(2,3)$, $D=(7,7)$, $E=(8,7)$. Run k-means with $k=2$ from initial centroids $c_1^{(0)}=(2,2)$, $c_2^{(0)}=(7,7)$. Show two complete iterations with assignments and updated centroids.

Q2 [Numerical · 10 marks · Intermediate] ** Six 2D points: $P_1=(0,0)$, $P_2=(1,0)$, $P_3=(0,1)$, $P_4=(5,5)$, $P_5=(6,5)$, $P_6=(5,6)$. Cluster using **single-linkage** agglomerative hierarchical clustering. Show the initial distance matrix, every merge, and the final dendrogram with heights.

Q3 [Numerical · 8 marks · Difficult] ** Same six points as Q2. Cluster using **complete-linkage**. State at which merge step (if any) the result diverges from the single-linkage answer, with reason.

Q4 [Numerical · 8 marks · Intermediate] ** A BFR DS cluster in 2D has $N = 20$, $\mathrm{SUM} = (40, 60)$, $\mathrm{SUMSQ} = (96, 200)$. A new point $x = (3, 4)$ arrives. (a) Find centroid and per-dimension σ. (b) Compute Mahalanobis distance. (c) With threshold $T = 2\sqrt{d}$, do we add x to the DS? (d) If yes, give the updated N, SUM, SUMSQ.

Q5 [Numerical · 6 marks · Intermediate] ** A CURE cluster has 4 points: $(0,0)$, $(4,0)$, $(0,3)$, $(4,3)$. Pick $r=2$ representatives by farthest-point rule. Shrink them by $\alpha = 0.2$ toward the centroid. Give the resulting reps.

Q6 [Numerical · 6 marks · Basic] ** Two CURE clusters. Cluster A reps: $(2.4, 2.4), (3.6, 0.6)$. Cluster B reps: $(10, 10), (12, 10)$. New point $p = (5, 5)$. Compute distance from p to every rep. Assign p to a cluster.

Q7 [Concept · 4 marks · Basic] ** Define centroid and clustroid. Why does k-means use centroids and hierarchical-on-non-Euclidean-space use clustroids? Give one example dataset where centroids are undefined.

Q8 [Concept · 4 marks · Intermediate] ** Compare single-linkage and complete-linkage agglomerative clustering on (a) the kinds of cluster shapes they prefer, (b) sensitivity to noise, (c) computational cost. Sketch a 1D example where the two give different 2-cluster outputs.

Q9 [Concept · 5 marks · Intermediate] ** State precisely the three sets BFR maintains and the meaning of each. Why is the discard set called "discarded"? What is the memory cost as a function of k and d?

Q10 [Concept · 5 marks · Difficult] ** Explain the Mahalanobis distance formula for a BFR cluster summarised by N, SUM, SUMSQ in dimension d. Why divide by σ_i in each dimension? State the threshold rule and justify why $\sqrt{d}$ appears.

Q11 [Concept · 5 marks · Difficult] ** Explain how CURE handles non-spherical clusters that BFR cannot. In your answer cover (a) representatives vs centroid, (b) shrinkage parameter α and what it protects against, (c) why Phase 2 assigns based on closest representative, not closest centroid.

Q12 [MCQ · 2 marks] ** In hierarchical agglomerative clustering with single-linkage, distance between two clusters A and B is: (A) maximum over all pairs (a, b), $a \in A$, $b \in B$ (B) minimum over all pairs (a, b), $a \in A$, $b \in B$ (C) distance between centroids (D) average pair distance

Q13 [MCQ · 2 marks] ** BFR stores per cluster: (A) every point in the cluster (B) just the centroid (C) N, SUM, SUMSQ — i.e. $2d + 1$ numbers (D) the full $d \times d$ covariance matrix and the centroid

Q14 [True/False · 2 marks] ** "k-means always finds the global optimum of the within-cluster sum of squares." Justify in one sentence.

Q15 [MCQ · 2 marks] ** A 2D BFR cluster has $N = 5$, $\mathrm{SUM} = (15, 10)$, $\mathrm{SUMSQ} = (49, 22)$. The centroid is: (A) (3, 2) (B) (15, 10) (C) (5, 5) (D) (45, 22)

---

§10 — Full Worked Answers

A1. Iteration 1 — Assignment.**

Pt	*d(·, (2,2))*	*d(·, (7,7))*	Assigned
A=(2,2)	0	*√50=7.07*	C1
B=(3,2)	1.00	*√16+25=6.40*	C1
C=(2,3)	1.00	*√25+16=6.40*	C1
D=(7,7)	*√50=7.07*	0	C2
E=(8,7)	*√36+25=7.81*	1.00	C2

C1 = *A,B,C*\, C2 = *D,E*\.

Iteration 1 — Update. $c_1^{(1)} = ((2+3+2)/3, (2+2+3)/3) = (7/3, 7/3) \approx (2.33, 2.33)$. $c_2^{(1)} = ((7+8)/2, (7+7)/2) = (7.5, 7)$.

Iteration 2 — Assignment. With centroids *(2.33, 2.33)* and *(7.5, 7)*: A, B, C still much nearer to C1; D, E still much nearer to C2. No reassignments.

Iteration 2 — Update. Centroids unchanged. **Converged.** Final clusters: *A,B,C*\ at *(2.33, 2.33)*, *D,E*\ at *(7.5, 7)*.

A2. Initial pairwise distances** (rounded to 2 d.p.):

	P1	P2	P3	P4	P5	P6
P1	0	1.00	1.00	7.07	7.81	7.81
P2		0	1.41	6.40	7.07	7.81
P3			0	6.40	7.81	7.07
P4				0	1.00	1.00
P5					0	1.41
P6						0

(Sample: $d(P_2,P_3)=\sqrt{1+1}=\sqrt{2} \approx 1.41$; $d(P_1,P_4)=\sqrt{25+25}=\sqrt{50} \approx 7.07$.)

Merge 1. Tie among $d(P_1,P_2)=1$, $d(P_1,P_3)=1$, $d(P_4,P_5)=1$, $d(P_4,P_6)=1$. Merge \P_1,P_2\ first at height *1*.

Update (single-linkage).

	{P1,P2}	P3	P4	P5	P6
{P1,P2}	0	1.00	6.40	7.07	7.81
P3		0	6.40	7.81	7.07
P4			0	1.00	1.00
P5				0	1.41
P6					0

Merge 2. $d(\{P_1,P_2\},P_3) = 1.00$ ties with $d(P_4,P_5)=1.00$ and $d(P_4,P_6)=1.00$. Merge $\{P_1,P_2,P_3\}$ at height *1.00*.

	{P1..P3}	P4	P5	P6
{P1..P3}	0	6.40	7.07	7.07
P4		0	1.00	1.00
P5			0	1.41
P6				0

(Single-linkage of {P1..P3} to P5: $min(7.81, 7.07, 7.81) = 7.07$.)

Merge 3. $d(P_4,P_5)=d(P_4,P_6)=1.00$. Merge $\{P_4,P_5\}$ at height *1.00*.

	{P1..P3}	{P4,P5}	P6
{P1..P3}	0	6.40	7.07
{P4,P5}		0	1.00
P6			0

Merge 4. $d(\{P_4,P_5\}, P_6)=1.00$. Merge $\{P_4,P_5,P_6\}$ at height *1.00*.

	{P1..P3}	{P4..P6}
{P1..P3}	0	6.40
{P4..P6}		0

Merge 5. Final merge at height *6.40*.

Dendrogram heights: *1, 1, 1, 1, 6.40*. Cut at any height in *(1.41, 6.40)* → 2 clusters $\{P_1,P_2,P_3\}$ and $\{P_4,P_5,P_6\}$.

A3. With complete-linkage** the initial 5 distance values are the same so Merge 1 is again $\{P_1,P_2\}$ at *1.00*. But the update changes:

After Merge 1: $d(\{P_1,P_2\},P_3)=max(1.00, 1.41)=1.41$ (single-linkage gave *1.00*).

So at Merge 2 the cheapest pair is now $d(P_4,P_5)=1.00$ (or $d(P_4,P_6)=1.00$) — *not* $d(\{P_1,P_2\},P_3)=1.41$. Merge $\{P_4,P_5\}$ at height *1.00*.

After Merges 1, 2, the matrix is:

	{P1,P2}	P3	{P4,P5}	P6
{P1,P2}	0	1.41	7.07	7.81
P3		0	7.81	7.07
{P4,P5}			0	1.41
P6				0

Cheapest is *1.41* at $d(\{P_1,P_2\},P_3)$ and $d(\{P_4,P_5\},P_6)$. Merge $\{P_1,P_2,P_3\}$ at height *1.41*, then $\{P_4,P_5,P_6\}$ at height *1.41*. Final merge at height *max(all 9 cross-cluster pairs) = 7.81*.

Divergence. The final cluster *composition* is the same: $\{P_1,P_2,P_3\}$, $\{P_4,P_5,P_6\}$. But the *dendrogram heights* differ — single-linkage gives the inner merges at height *1*, complete-linkage at height *1.41* (for the second tier) and *7.81* at the top instead of *6.40*. With more spread-out data the order of merges itself can flip.

A4. ** (a) Centroid: $c = (40/20, 60/20) = (2, 3)$. Variances: $\sigma_1^2 = 96/20 - 2^2 = 4.8 - 4 = 0.8$, so $\sigma_1 = 0.894$. $\sigma_2^2 = 200/20 - 3^2 = 10 - 9 = 1$, so $\sigma_2 = 1.000$.

(b) Mahalanobis distance from *(3, 4)*:

$$d_M = \sqrt{((3-2)/0.894)^2 + ((4-3)/1)^2} = \sqrt{1.250 + 1.000} = \sqrt{2.250} = 1.500$$

(c) Threshold $T\sqrt{d} = 2\sqrt{2} = 2.828$. Since $1.500 < 2.828$ → **accept**, add *x* to DS.

(d) Updated stats: $N = 21$. SUM = $(40+3, 60+4) = (43, 64)$. SUMSQ = $(96 + 9, 200 + 16) = (105, 216)$.

A5. ** Centroid: $c = ((0+4+0+4)/4, (0+0+3+3)/4) = (2, 1.5)$.

Distances to centroid: all four points are at $\sqrt{(2)^2 + (1.5)^2} = \sqrt{6.25} = 2.5$ — all tied. Pick lex first: $r_1 = (0, 0)$.

Distances from $r_1=(0,0)$:

- $(4,0)$: 4.000
- $(0,3)$: 3.000
- $(4,3)$: $\sqrt{16+9} = 5.000$

Farthest is $(4,3)$. So $r_2 = (4, 3)$.

Shrink with $\alpha = 0.2$ toward $c=(2, 1.5)$: $r_1' = (0,0) + 0.2 \cdot (2-0, 1.5-0) = (0.4, 0.3)$. $r_2' = (4,3) + 0.2 \cdot (2-4, 1.5-3) = (4, 3) + (-0.4, -0.3) = (3.6, 2.7)$.

Final reps: $\{(0.4, 0.3), (3.6, 2.7)\}$.

A6. ** Distances from $p=(5,5)$:

- to $(2.4, 2.4)$: $\sqrt{6.76 + 6.76} = \sqrt{13.52} \approx 3.677$
- to $(3.6, 0.6)$: $\sqrt{1.96 + 19.36} = \sqrt{21.32} \approx 4.617$
- to $(10, 10)$: $\sqrt{25 + 25} = \sqrt{50} \approx 7.071$
- to $(12, 10)$: $\sqrt{49 + 25} = \sqrt{74} \approx 8.602$

Closest rep is $(2.4, 2.4)$ in Cluster A. **Assign *p* to Cluster A.**

A7. Centroid** = the *mean* of all data-points in a cluster. Generally not itself a data-point — an artificial centre. **Clustroid** = the data-point in the cluster that is most central (e.g. minimises max distance to others, or sum of squared distances). Always a real data-point. k-means uses centroids because it lives in Euclidean space where averaging is well-defined; agglomerative clustering on non-Euclidean spaces (e.g. document-set Jaccard, DNA edit distance) cannot average two strings, so it picks a clustroid instead. Example with no centroid: Jaccard distance over user-rating sets — what's the "average" of two sets? Undefined.

A8. (a) Single-linkage** likes long, thin, chained clusters because it merges any two clusters that have *one* close pair. **Complete-linkage** likes compact, ball-like clusters — both "ends" of the merged cluster must be close. (b) Single-linkage is *very* sensitive to noise (one bridging point chains two unrelated clusters); complete-linkage is much more robust. (c) Both are $O(n^2 \log n)$ with priority queues — same big-O, similar constants.

1D example: Points at *0, 1, 2, 5, 6, 7* and a noise point at *3*. Single-linkage chains *\0,1,2,3,5,6,7* into one cluster via the bridge *3*. Complete-linkage keeps *\0,1,2* separate from *\5,6,7* because the max distance between them is large; the bridge point joins one or the other but doesn't fuse the groups.

A9. Discard set (DS) **— points definitively assigned to a cluster, replaced by summary stats and discarded.** Compression set (CS)** — small clusters of points that are close to each other but *not* close to any DS centroid; also stored as stats but not yet associated with any of the *k* clusters. **Retained set (RS)** — singleton outlier points, kept as raw points until either a CS forms around them or the algorithm ends.

"Discarded" because once a point is added to the DS we delete the actual coordinates; we keep only the cluster's *(N, SUM, SUMSQ)*. The point is gone from memory.

Memory cost: $O(k(2d+1))$ for DS plus $O(small)$ for CS and RS. Independent of n. For $k = 100$ clusters and $d = 100$ dimensions, only ~ *20*k numbers — fits trivially.

A10. **

$$d_M(x, c) = \sqrt{\sum_{i=1}^{d} \left(\frac{x_i - c_i}{\sigma_i}\right)^2}$$

Each dimension's deviation $(x_i - c_i)$ is normalised by *that dimension's own* standard deviation σ_i. This is required because BFR allows different per-dimension spreads — a *0.5*-unit deviation in a tight dimension ($\sigma = 0.1$) is huge (5σ); the same *0.5*-unit deviation in a loose dimension ($\sigma = 5$) is tiny (0.1σ). Plain Euclidean would treat them equally; Mahalanobis correctly weights them by significance.

Threshold rule. Under the assumption that points are normally distributed, after the Mahalanobis transform a point at one standard deviation in *every* dimension has $d_M = \sqrt{1^2 \cdot d} = \sqrt{d}$. So $\sqrt{d}$ is the natural unit of "1 sigma." Common rule: accept if $d_M < 2\sqrt{d}$ (2-sigma) or $3\sqrt{d}$ (3-sigma). The choice of T is the trade-off between false-positive (a stray point absorbed into the wrong cluster) and false-negative (a real cluster member rejected) errors.

A11. ** (a) BFR represents each cluster by a single centroid + per-dim variances → can only model axis-aligned ellipses. CURE represents each cluster by *r* scattered representatives → multiple "anchor points" can wrap around any shape — long curves, rings, S-curves. (b) The shrinkage *α* pulls each rep toward the cluster centroid. Without it a single noisy outlier could be picked as a rep and "capture" points actually belonging to a neighbouring cluster (because they are close to the outlier-rep). Shrinking by *20\%* moves the rep inside the cluster body, dampening the outlier's influence while still letting reps fan out across the shape. (c) Phase 2 assigns each new point *p* to the cluster whose *closest representative* is nearest. Using only the centroid would defeat the whole point — for a curved cluster, the centroid is in the empty interior and is *farther* from the cluster's own ends than from a different cluster.

A12. (B)**. Single-linkage = minimum pairwise distance.

A13. (C)**. N + SUM (d values) + SUMSQ (d values) = $2d + 1$.

A14. False.** k-means converges only to a *local* minimum of within-cluster SSE. The output depends on initialization. Typical practice: run with multiple random initializations and pick the best.

A15. Centroid = $(SUM_1/N, SUM_2/N) = (15/5, 10/5) = (3, 2)$. (A)**.

---

§11 — Ending Key Notes (Revision Cards)

Term	Quick-fact				
Cluster	Group with high intra-similarity and low inter-similarity.				
Euclidean distance	$\sqrt{\Sigma (x_i - y_i)^2}$ — for points in space.				
Cosine distance	$1 - \cos\theta$ — for vector direction (text).				
Jaccard distance	$1 -	A\cap B	/	A\cup B	$ — for sets.
Curse of dimensionality	In high d, all pairs are at nearly equal distance; clustering loses meaning.				
k-means update	Centroid $c_j = \frac{1}{	C_j	}\sum_{x\in C_j} x$.		
k-means convergence	Local minimum of within-cluster SSE; depends on initialization.				
Centroid	Mean of cluster — artificial point.				
Clustroid	Most-central data-point in cluster — actual point.				

Term	Quick-fact				
Single-linkage	$D(A,B) = min_{a,b}\ d(a,b)$ — chains, sensitive to noise.				
Complete-linkage	$D(A,B) = max_{a,b}\ d(a,b)$ — compact balls, noise-robust.				
Dendrogram	Binary tree of merges; cut at threshold to read off k.				
BFR sets	DS (discard, summarised), CS (compressed mini-clusters), RS (retained outliers).				
BFR summary stats	$(N, SUM, SUMSQ)$ — $2d+1$ numbers per cluster, additive across merges.				
BFR centroid	SUM_i/N in each dim.				
BFR variance	$\sigma_i^2 = SUMSQ_i/N - (SUM_i/N)^2$.				
Mahalanobis distance	$\sqrt{\Sigma_i}\ ((x_i - c_i)/\sigma_i)^2$ — per-dim normalised.				
Mahalanobis threshold	$T\sqrt{d}$ with $T = 2$ or 3 (2-sigma or 3-sigma rule).				
CURE phase 1	Sample → hier-cluster → pick r scattered reps → shrink reps toward centroid by α.				
CURE phase 2	Assign each remaining point to the cluster of its closest representative.				
Shrinkage α	Typical $\alpha = 0.2$. Moves reps inward to dampen outliers.				
BFR fails on	Tilted ellipses, non-convex clusters.				
CURE handles	Arbitrary cluster shapes via r reps + shrink.				
Mistake — out vs in linkage	Single = min, complete = max. Do not swap.				
Mistake — no σ_i	Forgetting per-dim normalisation in Mahalanobis = wrong distance.				

§12 — Formula & Algorithm Reference

Concept	Formula	When to use				
Euclidean distance	$d(x,y) = \sqrt{\Sigma_i}\ (x_i - y_i)^2$	k-means, BFR, CURE — any metric Euclidean question.				

Concept	Formula	When to use				
Cosine distance	$d(x,y) = 1 - x \cdot y / (\	x\	\	y\	)$	Text/document vectors.
Jaccard distance	$d(A,B) = 1 -	A\cap B	/	A\cup B	$	Set-based similarity.
k-means assignment	$\text{cluster}(x) = \arg\min_j \	x - c_j\	$	k-means inner loop.		
k-means update	$c_j = \frac{1}{	C_j	}\sum_{x \in C_j} x$	k-means inner loop.		
Single-linkage	$D(A,B) = min_{a,b}\ d(a,b)$	Hierarchical clustering — chained shapes.				
Complete-linkage	$D(A,B) = max_{a,b}\ d(a,b)$	Hierarchical clustering — compact shapes.				
Average-linkage	$D(A,B) = \frac{1}{	A		B	} \sum_{a,b} d(a,b) $	Compromise.
BFR centroid	$c_i = SUM_i / N$	Every BFR step.				
BFR variance	$\sigma_i^2 = SUMSQ_i/N - (SUM_i/N)^2$	Mahalanobis distance, merge decisions.				
Mahalanobis distance	$d_M(x,c) = \sqrt{\Sigma_i ((x_i - c_i)/\sigma_i)^2}$	BFR adding a new point.				
Mahalanobis threshold	Accept if $d_M < T\sqrt{d}$ (typical $T = 2$ or 3)	BFR membership test.				

Concept	Formula	When to use				
Stats merge (BFR/CS)	$N_{\cup} = N_1 + N_2$; SUM and SUMSQ add componentwise	Merging two clusters/CS sub-clusters cheaply.				
CURE rep shrink	$p' = p + \alpha (c - p)$ with $\alpha \approx 0.2$	CURE Phase 1 final step.				
CURE assignment (Phase 2)	$cluster(p) = argmin_C \, min_{r \in reps(C)} \, d(p, r)$	Phase 2 of CURE.				

Algorithmic complexity:

Algorithm	Per iter / pass	Memory	Scale
k-means	$O(nkd)$	$O(n + kd)$	RAM-bound
Hierarchical (naive)	$O(n^2)$ build, $O(n)$ merges → $O(n^3)$	$O(n^2)$	$n \lesssim 10^4$
Hierarchical (priority queue)	$O(n^2 \log n)$	$O(n^2)$	$n \lesssim 10^5$
BFR	$O(n)$ I/O, $O(\text{RAM-load} \cdot k\, d)$ compute	$O(kd)$ stats	Disk-resident
CURE	2 passes over n + sample-clustering	$O(\text{sample}^2 + krd)$	Disk-resident

Connections to other weeks:

- **W08-09 — PageRank:** spectral methods for graph clustering use the same eigenvector machinery, applied to the graph Laplacian.
- **W12 — Frequent Itemsets / SON:** the chunk-then-summarise pattern of BFR is the same data-engineering principle that drives SON/PCY frequent-itemsets algorithms.
- **MMDS book §7:** Chapter 7 covers all four algorithms with the same notation. §7.3 BFR, §7.4 CURE.

End of W13-14 Clustering exam-prep document.